

Manuel de l'Utilisateur

Compilateur DECAC

Projet Génie Logiciel

Équipe GL42



Grenoble INP - Ensimag
Janvier 2026

Résumé

Ce document constitue le manuel de référence pour le compilateur **decac**, développé dans le cadre du projet de Génie Logiciel à l'Ensimag par le groupe GL42. Il couvre l'installation, l'utilisation détaillée des commandes, la spécification du langage Deca, la gestion des erreurs et des exemples avancés de programmation.

Table des matières

1	Introduction	3
1.1	Présentation du projet	3
1.2	Architecture du compilateur	3
2	Installation et Configuration	4
2.1	Pré-requis système	4
2.2	Procédure d'installation	4
2.2.1	1. Récupération des sources	4
2.2.2	2. Compilation du projet	4
2.2.3	3. Configuration de l'environnement	4
2.3	Vérification de l'installation	5
3	Guide d'Utilisation	6
3.1	Ligne de commande	6
3.2	Options détaillées	6
3.2.1	Options d'information	6
3.2.2	Options de contrôle des étapes	6
3.2.3	Options de génération de code	6
3.2.4	Options avancées	7
3.3	Exemple de flux de travail	7
4	Référence du Langage Deca	8
4.1	Structure Lexicale	8
4.2	Types et Variables	8
4.2.1	Types primitifs	8
4.2.2	Déclaration	8
4.3	Structures de Contrôle	9
4.3.1	Conditionnelles	9
4.3.2	Boucles	9
4.4	Programmation Orientée Objet	9
4.4.1	Classes et Champs	9
4.4.2	Héritage	9
4.4.3	Polymorphisme et Appel Dynamique	9
4.4.4	Conversion de Types (Cast) et Instanceof	10
4.5	Entrées / Sorties	10

5	Gestion des Erreurs	11
5.1	Erreurs Syntaxiques	11
5.2	Erreurs Contextuelles	11
5.3	Erreurs à l'Exécution (Runtime)	11
6	Détails Techniques et Architecture Interne	12
6.1	Convention et Rôles des Registres	12
6.2	Stratégie d'Allocation des Registres	13
6.3	Architecture de Génération d'Expressions	13
6.3.1	API Publique	13
6.3.2	Implémentation Interne	13
6.4	Algorithme de Compilation Arithmétique	13
6.4.1	Cas Nominal (Registres Disponibles)	13
6.4.2	Cas Fallback (Spilling / Plus de registres)	14
6.5	Gestion de la Mémoire (Tas et Pile)	14
6.5.1	La Pile (Stack)	14
6.5.2	Le Tas (Heap)	14
7	Limitations et Perspectives	15
7.1	Limitations du Langage	15
7.2	Perspectives d'amélioration	15
8	Extension BYTE	16
8.1	Mode opératoire	16
8.1.1	Activation et compilation	16
8.1.2	Assemblage et exécution	16
8.2	Sous-ensemble du langage supporté	16
8.2.1	Couverture du langage	16
8.2.2	Compatibilité	17
8.3	Limitations et différences sémantiques	17
8.4	Bibliothèque standard	17
8.4.1	Correspondance des types	17
8.4.2	Classe Object	17
8.4.3	Entrées / Sorties	17
8.5	Messages d'erreur	17
	Annexes	19

Chapitre 1

Introduction

1.1 Présentation du projet

Le projet **Deca** consiste en la réalisation d'un compilateur complet pour un langage objet simplifié inspiré de Java, nommé *Deca*. Ce compilateur, nommé **decac**, traduit le code source Deca en langage assembleur pour une machine abstraite spécifique : **IMA** (Interactive Machine Architecture).

L'objectif principal est de fournir un outil robuste, capable de gérer les concepts fondamentaux de la programmation orientée objet : encapsulation, héritage et polymorphisme.

1.2 Architecture du compilateur

Le compilateur **decac** est structuré en plusieurs phases distinctes, assurant une séparation claire des responsabilités :

1. **Analyse Lexicale (Lexer)** : Découpage du texte source en lexèmes (mots-clés, identificateurs, littéraux). Utilisation de la bibliothèque ANTLR.
2. **Analyse Syntaxique (Parser)** : Vérification de la structure grammaticale et construction de l'Arbre Syntaxique Abstrait (AST).
3. **Analyse Contextuelle (Verify)** :
 - Décoration de l'arbre.
 - Vérification des types.
 - Gestion des portées et visibilité (environnements).
4. **Génération de Code (CodeGen)** :
 - Gestion de la mémoire (Pile et Tas).
 - Création de la table des méthodes virtuelles (VTable).
 - Traduction en instructions assembleur IMA.

Chapitre 2

Installation et Configuration

2.1 Pré-requis système

Le compilateur **decac** est écrit en Java et nécessite un environnement compatible.

- **Système d'exploitation** : Linux (recommandé), macOS, Windows (via WSL).
- **Java** : JDK version 17 ou supérieure (Testé sous JDK 23).
- **Maven** : Version 3.6 ou supérieure pour la gestion des dépendances.
- **IMA** : L'interpréteur de la machine abstraite doit être installé et accessible dans le PATH.

2.2 Procédure d'installation

2.2.1 1. Récupération des sources

Clonez le dépôt du projet depuis le serveur Git de l'école :

```
1 git clone https://gitlab.ensimag.fr/gl2026/gr7/gl42.git
2 cd projet-gl
```

2.2.2 2. Compilation du projet

Utilisez Maven pour compiler les sources et générer l'exécutable. La commande suivante nettoie les anciens fichiers, compile, lance les tests unitaires et construit le package :

```
1 mvn clean install
```

Si la compilation réussit, vous verrez un message **BUILD SUCCESS**.

2.2.3 3. Configuration de l'environnement

Pour utiliser **decac** depuis n'importe quel dossier, ajoutez le dossier **bin** à votre variable d'environnement **PATH**.

Sous Bash (**.bashrc**) ou Zsh (**.zshrc**) :

```
1 export PATH=$PATH:/chemin/vers/projet-gl/src/main/bin
```

Rechargez votre configuration :

```
1 source ~/.bashrc
```

2.3 Vérification de l'installation

Lancez la commande suivante pour vérifier que le compilateur est bien accessible :

```
1 decac -b
```

Vous devriez voir s'afficher : Deca compiler — GROUPE 7 / 42

Chapitre 3

Guide d'Utilisation

3.1 Ligne de commande

L'utilisation standard du compilateur se fait via le terminal. La syntaxe générale est :

```
1 decac [options] <fichier_source.deca> ...
```

Le compilateur accepte plusieurs fichiers sources en entrée et les traite séquentiellement (ou en parallèle avec l'option `-P`).

3.2 Options détaillées

Le comportement du compilateur peut être modifié grâce aux options suivantes :

3.2.1 Options d'information

- b (**Banner**) Affiche la bannière de l'équipe avec le nom des développeurs. Cette commande termine l'exécution immédiatement.

3.2.2 Options de contrôle des étapes

- p (**Parse**) Arrête le compilateur après l'étape d'analyse syntaxique. Affiche sur la sortie standard la décompilation du fichier source (le code source reconstitué à partir de l'arbre syntaxique). Utile pour vérifier comment le code est "compris" par le compilateur.
- v (**Verification**) Arrête le compilateur après l'étape d'analyse contextuelle. Cette option ne génère pas de fichier `.ass`. Elle permet de vérifier rapidement la validité des types et des déclarations sans attendre la génération de code.

3.2.3 Options de génération de code

- n (**No Check**) Désactive la génération des tests de débordement à l'exécution.
 - **Avantage** : Le code assembleur généré est plus court et plus rapide.
 - **Risque** : Les divisions par zéro, les débordements de pile ou les déréréférences de pointeurs nuls provoqueront des plantages brutaux de la machine IMA ou des résultats faux, sans message d'erreur explicite.

- r **X (Registers)** Limite le nombre de registres banalisés utilisés par l’allocateur de registres.
 - X doit être un entier compris entre 4 et 16.

3.2.4 Options avancées

- d (**Debug**) Active les traces de débogage internes. Affiche l’état des registres, les étapes de l’analyseur, et les graphes de flot de contrôle.
- P (**Parallel**) Active la compilation parallèle. Si plusieurs fichiers sont passés en argument, ils seront compilés simultanément sur plusieurs threads, accélérant le processus global sur les processeurs multi-cœurs.

3.3 Exemple de flux de travail

- 1. Écriture du fichier `test.deca`.
- 2. Vérification rapide : `decac -v test.deca`
- 3. Compilation complète : `decac test.deca` (génère `test.ass`)
- 4. Exécution : `ima test.ass`

Chapitre 4

Référence du Langage Deca

Le langage Deca est un langage typé statiquement, orienté objet. Cette section décrit sa syntaxe et sa sémantique.

4.1 Structure Lexicale

- **Commentaires** :
 - Ligne : `// commentaire`
 - Bloc : `/* commentaire */`
 - Documentation : `/** JavaDoc */` (supporté par le lexer mais ignoré par le parser).
- **Identificateurs** : Commencent par une lettre ou `_`, suivis de lettres, chiffres ou `_`. Sensibles à la casse.
- **Mots-clés réservés** : `class`, `extends`, `public`, `protected`, `int`, `float`, `boolean`, `void`, `if`, `else`, `while`, `return`, `this`, `new`, `null`, `instanceof`, `println`, `print`, `readInt`, `readFloat`.

4.2 Types et Variables

4.2.1 Types primitifs

- `int` : Entier signé 32 bits.
- `float` : Nombre flottant (IEEE 754 simple précision).
- `boolean` : Valeur logique (`true` ou `false`).

4.2.2 Déclaration

Les variables doivent être déclarées avec leur type. L'initialisation est optionnelle (valeur par défaut : 0, 0.0, false, null).

```
1 int x = 42;  
2 float pi = 3.14;  
3 boolean flag; // initialise a false
```

4.3 Structures de Contrôle

4.3.1 Conditionnelles

```
1 if (x > 0) {  
2     println("Positif");  
3 } else if (x < 0) {  
4     println("Négatif");  
5 } else {  
6     println("Nul");  
7 }
```

4.3.2 Boucles

Seule la boucle `while` est supportée.

```
1 while (x > 0) {  
2     x = x - 1;  
3 }
```

4.4 Programmation Orientée Objet

C'est le cœur du langage Deca.

4.4.1 Classes et Champs

```
1 class Point {  
2     protected int x; // Visible dans Point et ses sous-classes  
3     int y;           // public : Visible partout  
4  
5     void setX(int val) {  
6         this.x = val;  
7     }  
8 }
```

4.4.2 Héritage

Le mot-clé `extends` permet l'héritage simple. Toutes les classes héritent implicitement de la classe racine `Object` (qui fournit la méthode `equals`).

```
1 class Point3D extends Point {  
2     int z;  
3 }
```

4.4.3 Polymorphisme et Appel Dynamique

Les méthodes sont virtuelles par défaut. Le compilateur utilise une VTable pour résoudre l'appel à l'exécution en fonction du type réel de l'objet.

```
1 Point p = new Point3D();  
2 p.setX(10); // Appel correct meme si p est declare Point
```

4.4.4 Conversion de Types (Cast) et Instanceof

```
1 if (p instanceof Point3D) {  
2     Point3D p3 = (Point3D)(p);  
3     println(p3.z);  
4 }
```

Un cast invalide lève une erreur à l'exécution.

4.5 Entrées / Sorties

Des fonctions primitives permettent d'interagir avec l'utilisateur.

- `println(...)` : Affiche les arguments (chaînes, entiers, flottants) et un saut de ligne.
- `print(...)` : Affiche sans saut de ligne.
- `readInt()` : Lit un entier au clavier.
- `readFloat()` : Lit un flottant au clavier.

Chapitre 5

Gestion des Erreurs

Le compilateur `decac` a été conçu pour fournir des messages d'erreurs précis afin d'aider le développeur.

5.1 Erreurs Syntaxiques

Ces erreurs surviennent lorsque le code ne respecte pas la grammaire.

- **Format** : `<fichier>:<ligne>:<colonne>: message`
- **Exemple** : `test.deca:12:4: missing ';' at 'return'`
- **Cause** : Oubli de point-virgule, parenthèse fermante manquante, mot-clé mal orthographié.

5.2 Erreurs Contextuelles

Surviennent lors de la passe 1, 2 ou 3 (types, définitions). Exemples :

- **Identificateur inconnu** : Utilisation d'une variable non déclarée.
- **Type incompatible** : Tentative d'affecter un flottant à un entier sans cast, ou utilisation d'un booléen dans une addition.
- **Signature de méthode** : Redéfinition d'une méthode avec une signature différente dans une sous-classe.
- **Condition non booléenne** : Utilisation d'un entier dans un `if`.

5.3 Erreurs à l'Exécution (Runtime)

Ces erreurs arrêtent le programme assembleur en cours d'exécution.

STACK_OVERFLOW La pile est pleine. Cause fréquente : récursion infinie ou trop profonde.

HEAP_OVERFLOW Le tas est plein. Cause : trop d'allocations d'objets (`new`).

OVERFLOW_ARITHMETIQUE Débordement lors d'un calcul sur des flottants.

DIVISION_ZERO Division (ou modulo) par zéro.

DEREFERENCEMENT_NULL Tentative d'accès à un champ ou une méthode d'une variable valant `null`, non initialisée.

IO_ERROR Erreur de lecture lors d'un `readInt` ou `readFloat` (entrée mal formée).

Chapitre 6

Détails Techniques et Architecture Interne

Cette section détaille les choix d'implémentation bas niveau du compilateur `decac`, en particulier la gestion fine des registres et la stratégie de génération de code pour les expressions.

6.1 Convention et Rôles des Registres

Pour garantir la robustesse du code généré et simplifier la gestion des débordements (spilling), nous avons adopté une convention stricte de répartition des registres de la machine IMA.

La machine dispose de 16 registres généraux (R0 à R15). Notre compilateur les segmente comme suit :

R0 : Registre de Retour et Scratch

Réservé prioritairement pour le retour des méthodes (convention appelant/appelé). Il est également utilisé comme tampon de calcul temporaire ("scratch") lors d'opérations ne nécessitant pas de persistance.

R1 : Registre Résultat

C'est le registre "accumulateur" par défaut. Toute expression finale évaluée par le compilateur laisse son résultat dans R1. C'est également un registre scratch pour les calculs intermédiaires.

R2 : Registre de Secours (Spill Register)

Ce registre joue un rôle critique dans notre architecture :

- Il n'est **jamais** alloué par le `RegAllocator`.
- Il est utilisé exclusivement lors du "spilling" (manque de registres) pour récupérer des valeurs depuis la pile via `POP`.
- Choix architectural : Comme l'option `-r X` impose $X \geq 4$, R2 est garanti d'être toujours disponible et "stable", ce qui sécurise la génération de code dans les cas limites.

R3 ... Rmax : Registres Temporaires Allouables

Cette plage (allant de R3 à R15 ou moins selon l'option `-r`) est la seule gérée par le `RegAllocator`. Ils servent à stocker les résultats des sous-expressions.

- Dans les méthodes, ces registres sont sauvegardés (`PUSH`) et restaurés (`POP`) pour respecter la convention "Callee-saved".

6.2 Stratégie d'Allocation des Registres

Le composant `RegAllocator` a été conçu pour être léger et spécialisé.

- **Responsabilité Limitée** : Il ne gère que la plage `R3` à `Rmax`. Il ignore l'existence de `R0`, `R1` et `R2`.
- **Pas de choix final** : L'allocateur ne choisit jamais le registre de destination final d'une instruction (qui est souvent imposé à `R1` par l'appelant). Il sert uniquement à fournir des registres pour les opérandes temporaires (partie droite des opérations binaires).

6.3 Architecture de Génération d'Expressions

La génération de code des expressions repose sur une distinction claire entre l'API publique et l'implémentation interne, permettant une grande flexibilité.

6.3.1 API Publique

La méthode exposée aux autres nœuds de l'arbre est :

```
1 protected GRegister codeGenExpr(DecacCompiler compiler)
```

Contrat : Évalue l'expression complète et garantit que le résultat se trouve dans `R1`. Elle ne réalise aucune allocation persistante.

6.3.2 Implémentation Interne

La logique réelle est déléguée à une méthode abstraite :

```
1 protected abstract void codeGenExpr(DecacCompiler compiler, GRegister  
target)
```

Fonctionnement : Compile l'expression de sorte que le résultat atterrisse dans le registre `target` (généralement `R1`, mais peut être un registre temporaire alloué récursivement).

6.4 Algorithme de Compilation Arithmétique

Pour une opération binaire (ex : `A + B`), la classe `AbstractOpArith` implémente une stratégie à deux niveaux pour gérer le manque de registres.

Soit l'opération : `target = Left op Right`.

6.4.1 Cas Nominal (Registres Disponibles)

Si le `RegAllocator` peut fournir un registre libre :

1. Génération de `Left` dans `target` (souvent `R1`).
2. Allocation d'un registre temporaire (ex : `R3`).
3. Génération de `Right` dans `R3`.
4. Opération arithmétique : `OP R3, target`.
5. Libération de `R3`.

6.4.2 Cas Fallback (Spilling / Plus de registres)

Si tous les registres `R3..Rmax` sont occupés, nous utilisons la pile et le registre de secours `R2` :

1. Génération de `Left` dans `target`.
2. `PUSH target` : On sauvegarde le résultat de gauche sur la pile.
3. Génération de `Right` dans `target` (on réutilise le registre cible pour économiser).
4. `POP R2` : On récupère la valeur de gauche dans le registre de secours réservé.
5. Opération arithmétique : `OP target, R2`.
6. `LOAD R2, target` : On remet le résultat final dans la cible attendue.

```

1 ; Calcul de (ExpressionComplexe) + (AutreExpression)
2 ; Supposons R3..R15 pleins. Target = R1.
3
4 ; 1. Calcul Gauche
5 ... (code de gauche) ...
6 LOAD ... R1
7
8 ; 2. Sauvegarde
9 PUSH R1
10
11 ; 3. Calcul Droite (dans R1 pour economiser)
12 ... (code de droite) ...
13 LOAD ... R1
14
15 ; 4. Recuperation et Operation via R2
16 POP R2
17 ADD R1, R2
18 LOAD R2, R1

```

Listing 6.1 – Exemple de code assembleur généré (Cas Fallback)

Cette stratégie garantit que même les expressions arithmétiques les plus complexes (dépassant 16 niveaux d’imbrication) sont compilées correctement sans perte de données.

6.5 Gestion de la Mémoire (Tas et Pile)

6.5.1 La Pile (Stack)

Utilisée pour les variables locales et les paramètres. Chaque appel de méthode génère un bloc d’activation commençant par `TST0` (Test Stack Overflow) pour prévenir les débordements de pile infinis.

6.5.2 Le Tas (Heap)

Utilisé pour les objets instanciés par `new`.

- Structure d’un objet : `[Adresse VTable] [Champ 1] [Champ 2] ...`
- L’accès aux champs se fait via `LOAD d(Rx), Ry` où `Rx` contient l’adresse de l’objet (heap) et `d` est l’offset du champ déterminé à la compilation.

Chapitre 7

Limitations et Perspectives

Bien que fonctionnel, le compilateur Deca présente certaines limitations par rapport à Java standard.

7.1 Limitations du Langage

- **Pas de constructeurs** : L'initialisation doit se faire manuellement ou via des méthodes dédiées.
- **Pas de visibilité `private`** : Seuls `public` et `protected` sont supportés.
- **Pas de `static`** : Impossible de créer des constantes de classe ou des méthodes utilitaires sans instancier d'objet.
- **Pas de tableaux** : Les structures de données doivent être implémentées via des listes chaînées d'objets.
- **Pas de chaînes de caractères complexes** : Le type `string` ne sert qu'à l'affichage via `println`. Pas de concaténation ni de manipulation.

7.2 Perspectives d'amélioration

Des extensions futures pourraient inclure :

- L'ajout de la boucle `for`.
- L'optimisation du code généré (suppression du code mort, propagation de constantes).
- Le support des interfaces.

Chapitre 8

Extension BYTE

8.1 Mode opératoire

L’extension BYTE permet de compiler un programme Deca vers du bytecode Java exécutable sur la JVM.

8.1.1 Activation et compilation

L’extension est activée via l’option `--byte` du compilateur `decac`.

— **Commande :**

```
1 decac --byte <fichier.deca>
```

— **Résultat :** cette commande génère un fichier portant l’extension `.j` (format assembleur Jasmin) au lieu du fichier `.ass` standard.

8.1.2 Assemblage et exécution

Contrairement à la machine abstraite `ima`, le code produit pour la JVM nécessite des outils tiers pour être exécuté.

1. **Assemblage :**

```
1 java -jar jasmin.jar fichier.j
```

2. **Exécution :**

```
1 java <NomDeLaClasseMain>
```

8.2 Sous-ensemble du langage supporté

8.2.1 Couverture du langage

L’extension BYTE supporte l’intégralité du langage Deca, incluant la programmation orientée objet.

8.2.2 Compatibilité

Tout programme Deca valide syntaxiquement et contextuellement pour la machine IMA est également compilable vers la JVM.

8.3 Limitations et différences sémantiques

Bien que le langage source soit identique, l'exécution sur la JVM présente certaines différences techniques par rapport à la machine abstraite `ima`.

- **Arithmétique flottante** : la JVM applique strictement la norme IEEE 754. Certaines approximations ou affichages de flottants peuvent différer légèrement de l'exécution sur `ima`.
- **Gestion de la mémoire** : la JVM utilise un ramasse-miettes (Garbage Collector). Les erreurs de type `heap overflow` dépendent de la configuration de la JVM hôte et non d'une limite fixée par `decac`.
- **Pile d'exécution** : la gestion et les limites de la pile d'exécution peuvent différer entre la machine abstraite IMA et la JVM.

8.4 Bibliothèque standard

8.4.1 Correspondance des types

L'extension `BYTE` repose sur une correspondance directe entre les types primitifs Deca et ceux de la JVM :

- `int` $\rightarrow$ `I`
- `float` $\rightarrow$ `F`
- `boolean` $\rightarrow$ `Z`

8.4.2 Classe Object

La classe racine `Object` de Deca est mappée sur `java.lang.Object`. La méthode `equals` repose sur l'implémentation native de Java.

8.4.3 Entrées / Sorties

Les fonctions `print`, `println`, `readInt` et `readFloat` sont implémentées via la bibliothèque standard Java, notamment `System.out` et `java.util.Scanner`.

8.5 Messages d'erreur

En plus des erreurs standards du compilateur (lexicales, syntaxiques et contextuelles), l'extension `BYTE` peut produire les erreurs suivantes :

- **Erreur de génération Jasmin** : le fichier `.j` ne peut pas être écrit (droits insuffisants, disque plein).
- **Limites du format `.class`** : si une méthode dépasse la taille maximale autorisée (64 KB de bytecode), une erreur est signalée lors de l'assemblage avec Jasmin.

Contrairement à la machine abstraite IMA, qui produit des messages d'erreur d'exécution standardisés par le compilateur `decac`, l'extension `BYTE` délègue la gestion des erreurs d'exécution à la JVM.

Les erreurs sont alors signalées sous forme d'exceptions Java, avec les messages standards fournis par la machine virtuelle.

- Une division par zéro déclenche une exception `java.lang.ArithmeticException` (ex : `division par zero`).
- Un déréréfencement de `null` déclenche une `java.lang.NullPointerException`.
- Un transtypage invalide déclenche une `java.lang.ClassCastException`.

Annexes

A. Exemple de Code Assembleur Généré

Voici un extrait du code IMA généré pour une simple addition.
Code Deca :

```
1 {  
2     int x = 1;  
3     x = x + 2;  
4 }
```

Code Assembleur (.ass) :

```
1 ; start main program  
2 ; Beginning of main instructions:  
3     TST0 #3 ; Main Program  
4     ADDSP #3  
5 ; -----  
6 ; Construction des tables des methodes  
7 ; -----  
8     LOAD #null, R0  
9     STORE R0, 1(GB)  
10    LOAD code.Object.equals, R0  
11    STORE R0, 2(GB)  
12 ; -----  
13 ; Code du programme principal  
14 ; -----  
15    LOAD #1, R3  
16    STORE R3, 3(GB)  
17    LOAD 3(GB), R1  
18    LOAD #2, R3  
19    ADD R3, R1  
20    STORE R1, 3(GB)  
21    HALT  
22 code.Object.equals:  
23    LOAD -2(LB), R0  
24    CMP -3(LB), R0  
25    BEQ Object.equals.true  
26    LOAD #0, R0  
27    BRA Object.equals.end  
28 Object.equals.true:  
29    LOAD #1, R0  
30 Object.equals.end:  
31    RTS  
32 ; end main program
```